MethylForge — User Manual

A reproducible epigenome (DNA-methylation) analysis pipeline (methylation calls → imprinting · episignature · DMR → clinical & research reports)

Bright Adu

Genomics & Bioinformatics Core
Noguchi Memorial Institute for Medical Research
University of Ghana, Legon, Ghana

2026-07-30

Contents

1 Introduction **3**

2 Concepts **4**
- 2.1 DNA methylation and why the assay matters . . . 4
- 2.2 Bisulfite / EM conversion and its QC . . . 4
- 2.3 Imprinting-disorder analysis . . . 5
- 2.4 Episignatures . . . 5
- 2.5 Differential methylation (DMP / DMR) . . . 5

3 Pipeline architecture **6**
- 3.1 Modes . . . 6
- 3.2 Stages . . . 7
- 3.3 The Nextflow / profile model . . . 8

4 Installation **9**
- 4.1 Local install (Linux or macOS / Apple Silicon) . . . 9
- 4.2 Verifying the install . . . 9
- 4.3 Shared HPC install (SLURM + Apptainer/Singularity) . . . 10
- 4.4 How MethylForge uses conda envs . . . 10

5 The MethylForge command-line interface **11**

6 Inputs **12**
- 6.1 Sample sheet (Nextflow, `sequencing` mode) . . . 12
- 6.2 Methylation file formats (auto-detected) . . . 12
- 6.3 Reference genome . . . 12
- 6.4 Analysis resources . . . 13

7 Reference data & resources **14**
- 7.1 The imprinted-DMR panel . . . 14
- 7.2 Episignature references . . . 14
- 7.3 Bisulfite spike-in controls . . . 15
- 7.4 EPIC/450K manifest and array coordinates . . . 15

8 Configuration & compute resources **16**
- 8.1 Parameters . . . 16
- 8.2 Compute resources . . . 16

9 Running — step-by-step scenarios **18**
- 9.1 Quickstart on synthetic data . . . 18
- 9.2 Short-read bisulfite / EM-seq cohort . . . 18
- 9.3 Long-read native-5mC . . . 18
- 9.4 Pre-called methylation . . . 18
- 9.5 Array (Illumina EPIC / 450K) . . . 18
- 9.6 Batch QC only . . . 19

9.7 On the HPC . . . 19
9.8 Standalone subcommands (no alignment) . . . 19

10 Stage details **21**
10.1 Stage 0 — Array ingest (`array` mode) . . . 21
10.2 Stage 1 — Read QC & trimming (short) . . . 21
10.3 Stage 2 — Alignment . . . 21
10.4 Stage 3 — Methylation extraction (short) . . . 21
10.5 Stage 4 — Methylation QC . . . 21
10.6 Stage 5 — Imprinted-DMR analysis . . . 22
10.7 Stage 6 — Differential methylation . . . 22
10.8 Stage 7 — Episignature classification . . . 22
10.9 Stage 8 — Region annotation . . . 22
10.10 Stage 9 — Report engine and hand-off . . . 23
10.11 Batch cohort QC dashboard . . . 23

11 Outputs **24**

12 Quality control & interpretation **25**

13 Reproducibility & provenance **26**

14 Troubleshooting **27**

15 Limitations & validation **28**

16 Glossary **29**

17 Citation & the forge family **30**

Introduction

MethylForge is a reproducible, organism- and disease-agnostic pipeline for **DNA-methylation (epigenome) analysis**. It takes methylation data from any of the common assay families — short-read **bisulfite / EM-seq**, long-read **native 5mC** (Nanopore, PacBio HiFi), pre-called per-CpG methylation, or **Illumina EPIC/450K arrays** — normalises every input to a single per-cytosine schema, and produces:

- **methylation quality control** (bisulfite/EM conversion efficiency, coverage, global methylation by sequence context, inferred sex, pass/fail gates);
- **imprinted-DMR analysis** — loss/gain of methylation at a curated panel of clinically important imprinting control regions, mapped to the associated disorders, with a multi-locus imprinting disturbance (MLID) flag;
- **episignature classification** — reference-based genome-wide methylation classification against disorder-specific signatures, with a calibrated confidence and an explicit *inconclusive* zone;
- **differential methylation** — case-vs-control DMP/DMR calling with gene annotation;
- **branded clinical and research reports** — self-contained HTML with embedded figures, plus a machine-readable `results.json`, an Excel findings workbook, and a **FamilyForge-compatible** findings hand-off for downstream interpretation.

MethylForge is the **epigenome** member of the *forge* family. Its siblings are **CallForge** (targeted-capture germline sequence-variant calling) and **FamilyForge** (family-based variant interpretation and reporting). MethylForge emits the same `candidates.json`-compatible findings object CallForge does, so a methylation defect (for example an imprinting-locus loss of methylation, or a positive disorder episignature that supplies PS3-type functional evidence) can be carried into the same interpretation and report workflow as a sequence variant.

The engine has two layers. The **analysis + report core is pure Python** (`numpy/pandas/scipy/matplotlib/jin` and runs anywhere, including on an Apple-Silicon laptop, with no bioinformatics toolchain. The **upstream alignment / extraction stages are Nextflow DSL2 modules** that call the standard external tools (Bismark or bwa-meth, MethylDackel, minimap2/modkit, minfi), each in its own isolated conda environment.

> **Status note.** MethylForge ships with a deterministic synthetic fixture (`methylforge testdata`) that exercises every analysis stage and produces the *expected* biological result — a Prader-Willi imprinting call, a positive Sotos episignature, an implanted DMR, and an all-normal control. The fixture proves the **machinery**; it says nothing about performance on real cohorts. Panel coordinates, QC gates, and episignature classifiers **must be locally validated, ideally against ancestry-matched controls,** before any diagnostic use. Findings are research-grade until validated and must be interpreted by a qualified professional with orthogonal confirmation where indicated.

This manual documents what the code actually does, stage by stage, with generic example commands and the real input/output schemas.

Concepts

2.1 DNA methylation and why the assay matters

DNA methylation — the addition of a methyl group to cytosine, almost always in the **CpG** context in mammals — is a stable, heritable epigenetic mark that regulates imprinting, X-inactivation, and gene expression. MethylForge quantifies methylation as a **beta value** per cytosine: `beta = methylated_reads / total_reads`, a fraction from 0 (unmethylated) to 1 (fully methylated). A normal, biallelic, imprinted locus sits near `beta` $\approx$ `0.5` (one methylated allele); a fully methylated promoter sits near 1; an unmethylated CpG island near 0.

Different assays measure the same beta value by different means, and MethylForge supports the main families:

Assay family	Chemistry / signal	Upstream tools (in MethylForge)	Input to MethylForge
Short-read **bisulfite** (WGBS/RRBS)	unmethylated C → U (→ T) after bisulfite	FastQC, fastp, Bismark/bwa-meth, MethylDackel	paired FASTQ
Short-read **EM-seq**	enzymatic conversion (TET2+APOBEC), gentler	same short-read path	paired FASTQ
Long-read **native 5mC**	modified-base basecalling (MM/ML tags)	dorado (upstream, GPU), minimap2, modkit	modBAM / FASTQ
Array (EPIC/450K)	two-colour intensity at fixed probes	minfi/sesame (R)	IDAT → beta matrix
Pre-called	any of the above, already extracted	—	`.cov` / bedGraph / bedMethyl / TSV

Whatever the source, MethylForge reduces the data to one **normalised per-cytosine schema** (`chrom, start, end, context, meth, cov, beta`) so every downstream analysis is caller-agnostic.

2.2 Bisulfite / EM conversion and its QC

Both bisulfite and EM-seq work by converting unmethylated cytosine so it reads as thymine, while methylated cytosine is protected. The single most important quality metric for such an experiment is the **conversion efficiency** — the fraction of truly-unmethylated cytosines that were actually converted. Incomplete conversion inflates apparent methylation and can create false hyper-methylation calls. MethylForge estimates conversion, in order of preference, from:

1. **spike-in controls** aligned alongside the sample — unmethylated **lambda phage** DNA (residual methylation at lambda cytosines = non-conversion) and CpG-methylated **pUC19** (loss of methylation = *over*-conversion / false-negative rate);
2. a **provided** spike-in value;

3. **non-CpG (CHH) methylation** as a proxy (`1 - mean CHH beta`), since CHH is essentially unmethylated in most somatic tissue.

`bin/prepare_reference.sh` appends the spike-in FASTA to the reference so these controls align and are auto-detected by contig name (lambda `NC_001416`, pUC19 `L09137`, and common aliases).

2.3 Imprinting-disorder analysis

Imprinting disorders arise from aberrant methylation at **imprinting control regions** (ICRs) — loci where only one parental allele is normally methylated. MethylForge interrogates a curated panel (`resources/imprinted_dmrs.tsv`) of the clinically important ICRs and calls **hyper-** or **hypo-**methylation against a per-locus normal (≈ 0.5), mapping the *direction* of the defect to a disorder — e.g. *SNRPN* hypermethylation → Prader-Willi; *H19/IGF2* (IC1) hypomethylation → Silver-Russell; *KvDMR1* (IC2) loss → Beckwith-Wiedemann. When **two or more** loci are abnormal it flags **multi-locus imprinting disturbance (MLID)**. The bundled panel covers PWS/AS, BWS/SRS (IC1 and IC2), transient neonatal diabetes (6q24), Temple / Kagami-Ogata (14q32), pseudohypoparathyroidism (GNAS), and the MEST/GRB10 SRS loci.

2.4 Episignatures

Many Mendelian disorders — particularly those affecting the epigenetic machinery (chromatin writers/readers/erasers) — produce a reproducible, genome-wide **episignature**: a characteristic pattern of methylation at a specific set of CpGs. Following the Aref-Eshghi/Sadikovic framework, MethylForge classifies a query methylation profile against a **reference** of disorder-specific signatures. The default classifier is a dependency-free **nearest-centroid / correlation** model (so it runs anywhere); a **scikit-learn linear SVM** is used automatically when scikit-learn is installed. A call is made only when the top class both clears a confidence threshold and beats the runner-up by a margin — otherwise the verdict is **inconclusive**, so uncertain samples are never over-called. A positive disorder episignature can serve as **functional (PS3-type) evidence** for a related sequence variant. MethylForge also *builds and validates* references from a labelled cohort matrix with nested leave-one-out cross-validation (`methylforge build-reference`).

2.5 Differential methylation (DMP / DMR)

For cohort studies, MethylForge calls **differentially methylated positions (DMPs)** between two groups (case vs control) on a per-CpG beta matrix, then merges adjacent, same-direction DMPs into **differentially methylated regions (DMRs)**. Per-CpG testing uses a Mann-Whitney U test for larger cohorts and a Welch t-test on **M-values** (the logit transform of beta, the standard for parametric methylation testing) for small cohorts where the rank test's p-floor is too high to survive multiple-testing correction. Correction is **Benjamini-Hochberg FDR**. DMRs can be annotated with the nearest gene from a BED file.

Pipeline architecture

MethylForge routes each sample from its input type to the normalised per-cytosine methylation schema, then runs the shared analysis + report engine. The Nextflow layer handles the heavy per-sample alignment/extraction and orchestration; the Python engine handles QC, imprinting, episignature, DMR, reporting and the hand-off.

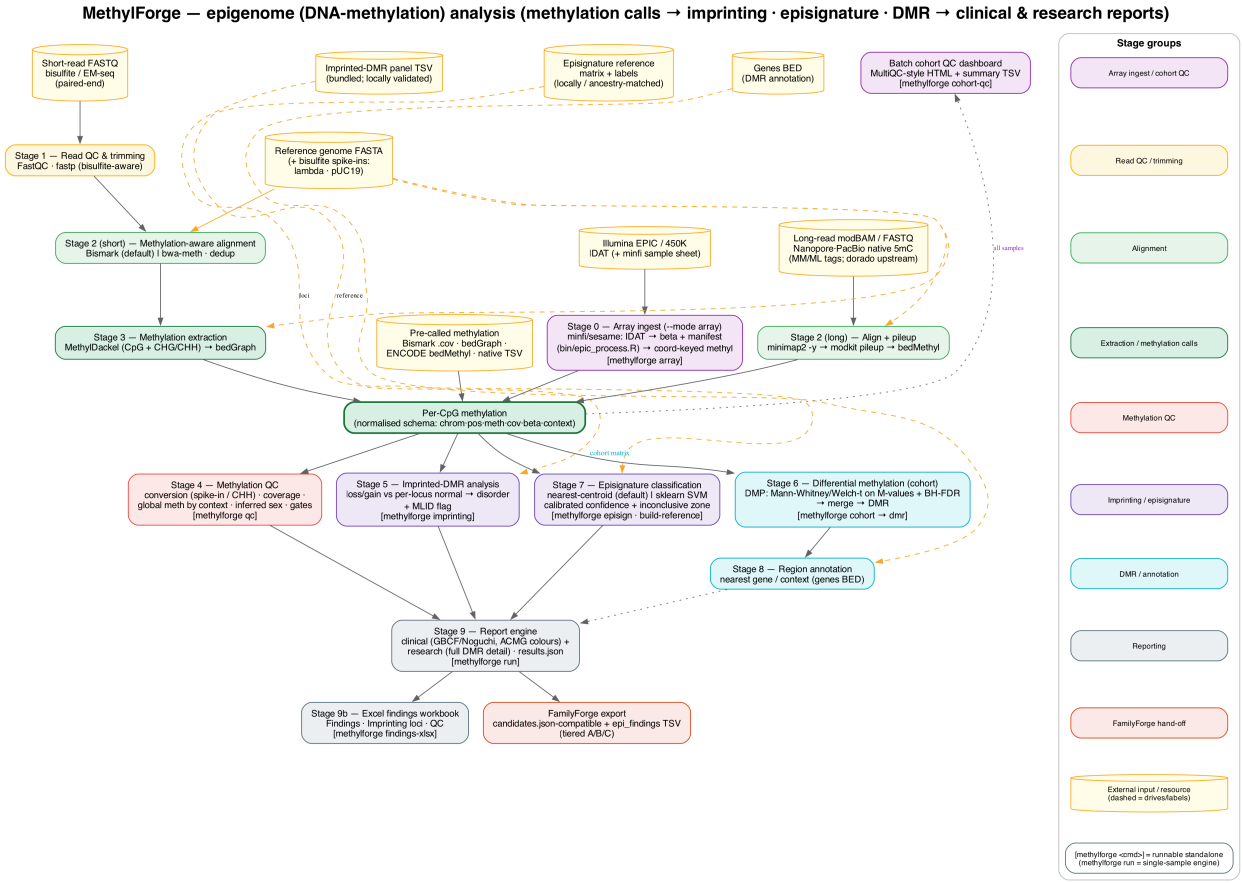


Figure 3.1: MethylForge architecture: external inputs/resources (yellow cylinders) feed the input-specific paths (array ingest, short-read QC→align→extract, long-read align→pileup, or pre-called), which converge on the normalised per-CpG methylation schema; the analysis engine (QC, imprinting, episignature, DMR/annotation) then feeds the report engine and the FamilyForge hand-off. Bracketed `[methylforge <cmd>]` labels mark the runnable standalone subcommands.

3.1 Modes

The pipeline entry point (`main.nf`) selects one of three **modes** with `--mode`:

Mode	Trigger	What it runs
`sequencing` (default)	`--samplesheet`	per-sample short/long/pre-called → analysis + reports (+ batch QC)
`array`	`--idat_dir`+`--array_sheet`, or `--array_beta`+`--array_manifest`	IDAT → beta (minfi) → per-sample methylation → analysis + reports
`cohort_qc`	`--samplesheet` (of methylation files)	batch QC dashboard only

Within `sequencing`, each sample's `read_type` (`short` | `long` | `precalled`) sends it down the appropriate path; the three paths converge and share every downstream stage.

3.2 Stages

#	Stage	Key tools / module	Standalone command	Output (per sample unless noted)
0	Array ingest (`array` mode)	minfi/sesame · `bin/epic_process.R` · `array.py`	`methylforge array`	`results/array/…` beta + manifest + per-sample `*.methyl.tsv`
1	Read QC & trimming (short)	FastQC, fastp	—	`<sample>/qc/…` trimmed FASTQ + `fastp.json`
2s	Alignment (short)	Bismark (default) | bwa-meth, dedup	`methylforge align`*	`<sample>/align/<sample>.dedu`
2l	Alignment + pileup (long)	minimap2 `-y`, modkit	—	`<sample>/align/<sample>.cpg`
3	Methylation extraction (short)	MethylDackel	—	`<sample>/methylation/<sample`
4	Methylation QC	`qc.py`	`methylforge qc`	metrics in `results.json` + report
5	Imprinted-DMR analysis	`imprinting.py`	`methylforge imprinting`	imprinting table + summary
6	Differential methylation	`cohort.py` + `dmr.py`	`methylforge cohort` → `dmr`	DMP table + merged DMRs
7	Episignature classification	`episignature.py`	`methylforge episign` / `build-reference`	verdict + probabilities

#	Stage	Key tools / module	Standalone command	Output (per sample unless noted)
8	Region annotation	`annotate.py`	(in `dmr/run` via `--genes-bed`)	nearest-gene column on DMRs
9	Report engine	`report.py` (jinja2)	`methylforge run`	clinical + research HTML, `results.json`, `.findings.xlsx`
9b	FamilyForge / Excel hand-off	`export_familyforge.py`, `export_excel.py`	`methylforge findings-xlsx`	`<sample>.familyforge.json` + `epi_findings.tsv`
—	Batch cohort QC	`dashboard.py`	`methylforge cohort-qc`	`cohort_qc.html` + `cohort_qc_summary.tsv` (per run)

* The Nextflow module names are `QC_TRIM`, `ALIGN_SHORT`, `ALIGN_LONG`, `EXTRACT`, `ANALYSE_REPORT`, `COHORT_QC`, `ARRAY_INGEST`, `ARRAY_TO_METHYL`; the bracketed `[methylforge …]` names in the diagram are the equivalent Python-CLI subcommands you can run standalone on already-called methylation.

3.3 The Nextflow / profile model

`nextflow.config` defines **two composable profile groups** (mirroring CallForge):

- **execution**: `local` | `hpc_slurm`
- **packaging**: `conda` | `mamba` | `docker` | `singularity` | `apptainer`

plus a `test` profile. You compose one from each group, e.g. `-profile local,conda` on a laptop or `-profile hpc_slurm,singularity` on a cluster. Each process is tagged with a **label** (`qc_trim`, `align_short`, `align_long`, `extract`, `analysis`, `report`, `array`) that maps in `conf/base.config` to a per-stage conda environment and resource request.

> **Classic-DSL2 syntax.** The modules use Nextflow's v1 language parser. Export `NXF_SYNTAX_PARSER=v1` once per shell before running (recent Nextflow defaults to a stricter parser). The examples below assume you have done so.

Installation

MethylForge has a lightweight core (pure-Python analysis + reports) and optional heavy envs (aligners) that are only needed if you start from raw reads or IDATs.

4.1 Local install (Linux or macOS / Apple Silicon)

```
git clone https://github.com/aduton1000/methylforge.git
cd methylforge

# 1) core analysis + report engine (pure Python; solves on any platform)
mamba env create -f env/methylforge.yml
conda activate methylforge
pip install -e .            # run this from the REPO ROOT
methylforge version
```

That is enough to run everything that operates on **already-called** methylation: `run`, `qc`, `imprinting`, `dmr`, `episign`, `build-reference`, `array`, `cohort`, `cohort-qc`, `findings-xlsx`, and the whole synthetic demo.

To start from **raw reads or IDATs**, create the per-stage envs as well:

```
# short-read bisulfite/EM path (FastQC, fastp, Bismark, bwa-meth, MethylDackel,
↪  samtools)
mamba env create -f env/align_short.yml
mamba env create -f env/extract.yml
# long-read native-5mC path (minimap2, modkit)
mamba env create -f env/align_long.yml
# array path (R/Bioconductor: minfi + EPIC/450K manifests)
mamba env create -f env/array.yml
```

> **Apple Silicon.** The core `methylforge` env is pure Python and installs **natively** (arm64). The heavy-aligner envs pull bioconda binaries that are only built for `osx-64`; create those under Rosetta: `CONDA_SUBDIR=osx-64 mamba env create -f env/align_short.yml`. Linux/HPC is native throughout. ONT `dorado` basecalling (GPU, modified-base models) is done **upstream** and is distributed by ONT separately — it is not a conda package; MethylForge consumes the resulting modBAM.

> **The array env solver.** Mixing `sesame` (tracks the newest R) with the older EPIC annotation packages can be unsolvable on some platforms. `env/array.yml` therefore installs minfi + the manifests (which resolve everywhere) and leaves `sesame` optional. Without sesame, `bin/epic_process.R` returns hg19 coordinates unless you pass an hg19→hg38 liftOver chain (`--array_chain`); see *Reference data*.

4.2 Verifying the install

```
# programmatic assertion suite (expected calls on synthetic data)
python tests/selftest.py
```

```
# end-to-end demo: PWS / Sotos / control through the analysis + report engine
methylforge testdata -o mf_demo
bash test/run_demo.sh mf_demo
```

The self-test generates the fixture, runs the pipeline, and asserts the expected Prader-Willi imprinting call, positive Sotos episignature, an implanted DMR, the report HTMLs, and the Excel findings workbook. `test/smoke_shortread.sh` and `test/test_array_hpc.sh` additionally exercise the real FASTQ→Bismark→MethylDackel and IDAT→beta toolchains (they need the heavy envs).

4.3 Shared HPC install (SLURM + Apptainer/Singularity)

Full instructions are in `docs/hpc_deployment.md`. In brief:

```
cd /opt/lab/pipelines            # a path all users can read
git clone https://github.com/aduton1000/methylforge.git
cd methylforge
mamba env create -f env/methylforge.yml && conda activate methylforge && pip
 ↪  install -e .

export NXF_SYNTAX_PARSER=v1
export NXF_SINGULARITY_CACHEDIR=/opt/lab/singularity_cache
nextflow run main.nf -profile hpc_slurm,singularity \
  --samplesheet samples.csv --reference /path/to/GRCh38.fa
```

`nextflow.config` already defines `hpc_slurm` (executor=slurm, queue chosen by task time, `queueSize=100`). Adjust `process.queue`, `--max_cpus`, `--max_memory`, and the per-label resources in `conf/base.config` to your partitions. The deployment guide also shows a one-command lab wrapper (`bin/methylforge-run`) that bakes in the module loads, the syntax-parser flag, the container cache, and the shared reference paths, so a user runs a single command.

4.4 How MethylForge uses conda envs

Each Nextflow process declares its own environment via `conda = "…/env/<x>.yml"` in `conf/base.config`, so the toolchains are **isolated** (Bismark's bowtie2 never collides with modkit's dependencies, etc.). The `methylforge` package itself is installed **once** into the core env and called from the `analysis`/`report`/`array` processes. Under a container profile (`singularity`/`apptainer`/`docker`) Nextflow provisions each process from an image; under `conda`/`mamba` it builds the env on first use and caches it.

The MethylForge command-line interface

`methylforge` is a single entry point with additive subcommands. Every stage is runnable standalone on already-called methylation, so you can re-QC, re-annotate, or re-classify without re-running alignment.

Subcommand	Purpose	Key required inputs
`run`	end-to-end single-sample analysis → clinical + research HTML, `results.json`, `.findings.xlsx`, FamilyForge export	`methylation`, `-s/--sample`
`qc`	methylation QC only (JSON)	`methylation`
`imprinting`	imprinted-DMR analysis only	`methylation`
`dmr`	cohort differential methylation	`--matrix`, `--groups`
`cohort`	build a feature × sample beta matrix from many samples	`--samplesheet/--inputs`, `--out`
`cohort-qc`	MultiQC-style batch QC dashboard	one of `--samplesheet/--inputs/--results`, `--out`
`episign`	classify a query against a reference	`--reference`, `--labels`, `--query`
`build-reference`	build + LOO-validate an episignature reference	`--matrix`, `--labels`, `-o`
`array`	ingest an EPIC/450K beta matrix → coord matrix / per-sample methyl	`--beta`
`findings-xlsx`	aggregate per-sample FamilyForge findings into one workbook	`--familyforge`, `--out`
`testdata`	generate the synthetic fixture	—
`version`	print version	—

```
methylforge --help          # top-level
methylforge run --help      # per-subcommand options
```

Inputs

6.1 Sample sheet (Nextflow, `sequencing` mode)

A CSV with a header. Columns (unused columns may be blank; `read_type` may be set per-row or defaulted with `--read_type`):

```
sample,fastq_1,fastq_2,read_type,modbam,methylation,group
S1,S1_R1.fq.gz,S1_R2.fq.gz,short,,,case
S2,,,long,S2.mod.bam,,case
S3,,,precalled,,S3.cov.gz,control
```

Column	Meaning	Used when
`sample`	unique sample id	always
`fastq_1`, `fastq_2`	paired short-read FASTQ	`read_type=short`
`read_type`	`short` | `long` | `precalled`	per-row (else `--read_type`)
`modbam`	long-read modBAM (MM/ML) or FASTQ	`read_type=long`
`methylation`	pre-called `.cov`/bedGraph/bedMethyl/TSV	`read_type=precalled`
`group`	`case`/`control` (or any label) for cohort DMR	optional

`test/samplesheet_precalled.csv` is a working three-sample pre-called example.

6.2 Methylation file formats (auto-detected)

`io.read_methylation` sniffs the format by extension/header and normalises to `chrom, start, end, context, meth, cov, beta`:

- **Bismark coverage** `.cov`/`.cov.gz` — `chrom start end pct meth unmeth`;
- **bedGraph** `.bedGraph` — 6-column MethylDackel (carries real coverage) or plain 4-column (percentage only; a nominal coverage of 30 is assigned);
- **ENCODE bedMethyl** `.bed`/`.bedmethyl` (modkit / ONT) — valid-coverage and modified counts columns;
- **generic/native TSV** — any file with `chrom`,`start` and (`beta` or `meth`+`cov`).

6.3 Reference genome

For the short/long sequencing paths supply `--reference /path/to/genome.fa`. Bismark builds (and reuses) its bisulfite index from the genome's directory on first run; bwa-meth builds its own index; long-read uses the FASTA directly with minimap2 and modkit. To enable spike-in conversion QC, prepend the controls first:

```
bin/prepare_reference.sh /path/to/GRCh38.fa /path/to/spikein_controls.fa \
  /path/to/GRCh38.withspikein.fa
```

6.4 Analysis resources

Resource	Param / flag	Notes
Imprinted-DMR panel	`--imprint_panel` (`--panel` in CLI)	bundled `resources/imprinted_dmrs.tsv`; **validate locally**
Episignature reference	`--episign_ref` + `--episign_labels`	build with `build-reference`; ancestry-matched
Genes BED	`--genes_bed`	`chrom start end name`; DMR nearest-gene annotation
EPIC/450K manifest	`--array_manifest` (or built by `epic_process.R`)	probe→coordinate; user-supplied (licensed)
liftOver chain	`--array_chain`	hg19→hg38 fallback for array coordinates

Reference data & resources

7.1 The imprinted-DMR panel

`methylforge/resources/imprinted_dmrs.tsv` is a tab-separated table with columns `locus`, `gene`, `chrom`, `start`, `end`, `normal_meth`, `hyper_disorder`, `hypo_disorder`, `notes`. Each row is an ICR with a per-locus normal methylation (≈ 0.50 for a biallelic imprinted region) and the disorder associated with each *direction* of aberration. The bundled loci:

Locus	Gene	Region (GRCh38)	Hyper →	Hypo →
PWS/AS-IC	SNRPN	chr15:24.95–24.96 Mb	Prader-Willi	Angelman
IC1 (H19/IGF2)	H19	chr11:2.00–2.00 Mb	Beckwith-Wiedemann	Silver-Russell
IC2 (KvDMR1)	KCNQ1OT1	chr11:2.70–2.70 Mb	—	Beckwith-Wiedemann
6q24	PLAGL1	chr6:144.01 Mb	—	Transient neonatal diabetes
14q32	MEG3	chr14:100.82 Mb	Kagami-Ogata	Temple
GNAS	GNAS	chr20:58.84 Mb	Pseudohypoparathyroidism	Pseudohypoparathyroidism
MEST	MEST	chr7:130.49 Mb	—	Silver-Russell
GRB10	GRB10	chr7:50.78 Mb	—	Silver-Russell (UPD7)

> **Coordinates are a starting reference.** They are GRCh38-based and must be confirmed against your reference build and, ideally, re-centred on your own normal ranges before diagnostic use. Provide a replacement panel with the same columns via `--imprint_panel`.

7.2 Episignature references

An episignature reference is two files: a **feature × sample beta matrix** (`--episign_ref`, features indexed as `chrom:pos`) and a **`sample<TAB>class` label map** (`--episign_labels`, one class must be `Control`). Build one from a labelled cohort matrix:

```
methylforge build-reference \
  --matrix cohort_beta.tsv --labels cohort_labels.tsv \
  --features-per-class 40 --min-delta 0.15 --alpha 0.05 \
  -o refs/my_episig
```

This selects the most discriminative CpGs per non-control class ($|\Delta\beta| \geq$ `min-delta` **and** M-value Welch-t p $<$ `alpha`, top N by $|\Delta\beta|$), builds class centroids, and runs **nested leave-one-out cross-validation** (features re-selected on each training fold — no leakage), reporting per-class sensitivity/specificity. It writes `reference_matrix.tsv`, `labels.tsv`, `centroids.tsv`, `signatures.json`, and a `manifest.json` with the CV metrics. The first two are drop-in for `--episign-ref/--episign-labels`.

7.3 Bisulfite spike-in controls

To measure conversion from the data, append unmethylated **lambda** and CpG-methylated **pUC19** to the reference (`bin/prepare_reference.sh`) so their reads align. QC auto-detects them by contig name (lambda `NC_001416/chrL/J02459…`, pUC19 `L09137/M77789…`). Without spike-ins, conversion falls back to CHH methylation.

7.4 EPIC/450K manifest and array coordinates

MethylForge does **not** bundle the licensed Illumina manifest. `bin/epic_process.R` derives probe→coordinate mapping from minfi's annotation (hg19) and remaps to hg38 by trying, in order: sesame's hg38 manifest, then a deterministic UCSC hg19→hg38 liftOver (pass the chain via `--array_chain`), else it keeps hg19 and says so in `epic_qc.txt`. For the Python `array` subcommand you supply the manifest with `--manifest` (tolerant of `Name`/`IlmnID`, `chr`/`CHR`, `pos`/`MAPINFO` column names).

Configuration & compute resources

8.1 Parameters

Set on the command line (`--param value`) or in a profile/config. The full set from `nextflow.config`:

Parameter	Default	Meaning
`mode`	`sequencing`	`sequencing` | `array` | `cohort_qc`
`samplesheet`	`null`	input CSV (sequencing / cohort_qc)
`outdir`	`results`	output directory
`reference`	`null`	genome FASTA (sequencing paths)
`targets`	`null`	optional BED for targeted/capture methyl-seq
`read_type`	`short`	default read type (per-row column overrides)
`library`	`bisulfite`	`bisulfite` | `emseq` conversion chemistry
`imprint_panel`	bundled TSV	imprinted-DMR panel
`episign_ref` / `episign_labels`	`null`	episignature reference + labels
`genes_bed`	`null`	gene BED for DMR annotation
`run_dmr`	`false`	cohort DMR (needs a group column + matrix/groups)
`cohort_qc`	`true`	batch QC dashboard at the end of a run
`report_modes`	`clinical,research`	which reports to render
`idat_dir` / `array_sheet`	`null`	array-mode IDAT inputs
`array_genome`	`hg38`	coordinate build for the manifest
`array_chain`	`null`	hg19→hg38 liftOver chain
`array_beta` / `array_manifest`	`null`	pre-computed beta (skip the R step)
`max_cpus`	`16`	CPU cap per process
`max_memory`	`120.GB`	memory cap
`max_time`	`48.h`	time cap

8.2 Compute resources

The base `process` block requests `4` × `attempt` CPUs (capped at `max_cpus`), `8.GB` × `attempt` memory, and `4.h` × `attempt` time, retrying twice on out-of-memory / signal exit codes

(137/139/140/143/247). Per-stage overrides live in `conf/base.config` by label:

Label	Env	CPUs	Memory	Time
`qc_trim`	align_short	4	8 GB	—
`align_short`	align_short	min(8, max_cpus)	32 GB	12 h
`align_long`	align_long	min(8, max_cpus)	32 GB	12 h
`extract`	extract	4	16 GB	—
`analysis`	methylforge	2	8 GB	—
`report`	methylforge	1	4 GB	—

On a laptop, cap the whole run with `--max_cpus 8 --max_memory 24.GB`. WGBS is data-heavy: give runs generous scratch and `nextflow clean -f` the `work/` directory between large runs.

Running — step-by-step scenarios

All Nextflow examples assume `export NXF_SYNTAX_PARSER=v1` in the current shell.

9.1 Quickstart on synthetic data

```
methylforge testdata -o mf_demo          # synthetic methylation for 3 samples +
↪  cohort + array
bash test/run_demo.sh mf_demo            # PWS / Sotos / control end-to-end
python tests/selftest.py                 # programmatic assertions
```

Expected: **PWS case → Prader-Willi (SNRPN hypermethylation)**, **Sotos case → positive episignature + a DMR**, **control → all normal**.

9.2 Short-read bisulfite / EM-seq cohort

```
nextflow run main.nf -profile local,conda --mode sequencing \
  --samplesheet samples.csv --reference /path/to/GRCh38.withspikein.fa \
  --library bisulfite \
  --episign_ref /path/to/episig_reference.tsv \
  --episign_labels /path/to/episig_labels.tsv \
  --outdir results
```

Each `short` sample runs FastQC + fastp → Bismark (or bwa-meth) + dedup → MethylDackel → analysis + reports. Choose the aligner with `--aligner bwameth` (default `bismark`).

9.3 Long-read native-5mC

```
nextflow run main.nf -profile local,conda --mode sequencing \
  --samplesheet samples.csv --reference /path/to/GRCh38.fa
```

`long` samples (modBAM with MM/ML tags, or FASTQ) are aligned with `minimap2 -y` (preserving modification tags) and piled up with `modkit` to a per-CpG bedMethyl, then flow through the identical analysis. Basecalling with modified-base models (ONT `dorado`, GPU) is done upstream.

9.4 Pre-called methylation

```
nextflow run main.nf -profile local,conda --mode sequencing \
  --samplesheet test/samplesheet_precalled.csv --run_dmr false
```

`precalled` samples skip straight to analysis — useful for re-reporting existing Bismark/modkit output.

9.5 Array (Illumina EPIC / 450K)

```
# from IDATs (runs minfi in R, then the shared analysis)
```

```
nextflow run main.nf -profile local,conda --mode array \
  --idat_dir /path/to/idats --array_sheet /path/to/minfi_sheet.csv \
  --array_genome hg38 --array_chain /path/to/hg19ToHg38.over.chain

# from a pre-made beta matrix (skips the R step)
nextflow run main.nf -profile local,conda --mode array \
  --array_beta epic_beta.tsv --array_manifest epic_manifest.tsv
```

9.6 Batch QC only

```
nextflow run main.nf -profile local,conda --mode cohort_qc \
  --samplesheet methyl_sheet.csv        # CSV: sample,methylation
```

Every `sequencing/array` run **also** emits `cohort_qc.html` automatically; disable with `--cohort_qc false`.

9.7 On the HPC

```
nextflow run main.nf -profile hpc_slurm,singularity --mode sequencing \
  --samplesheet samples.csv --reference /path/to/GRCh38.fa \
  --outdir /scratch/$USER/methyl_run
```

9.8 Standalone subcommands (no alignment)

Re-run just the piece you need on already-called methylation:

```
# single-sample end-to-end (the engine the Nextflow analysis stage calls)
methylforge run sample.cpg.bedGraph -s SAMPLE --family FAM01 \
  --panel /path/to/imprinted_dmrs.tsv \
  --episign-ref refs/episig_reference.tsv --episign-labels refs/episig_labels.tsv
  ↪  \
  -o out/SAMPLE

# QC / imprinting only
methylforge qc sample.cov.gz
methylforge imprinting sample.cov.gz --panel /path/to/imprinted_dmrs.tsv

# cohort DMR: build a matrix, then call DMPs/DMRs
methylforge cohort --samplesheet cohort.csv --out beta_matrix.tsv --groups-out
 ↪  groups.tsv
methylforge dmr --matrix beta_matrix.tsv --groups groups.tsv --genes-bed
 ↪  genes.bed

# episignature classification against an existing reference
methylforge episign --reference refs/reference_matrix.tsv \
  --labels refs/labels.tsv --query query_beta.tsv

# aggregate cohort findings into one Excel workbook
```

```
methylforge findings-xlsx --familyforge 'out/*/*.familyforge.json' --out
↪  cohort_findings.xlsx
```

Stage details

10.1 Stage 0 — Array ingest (`array` mode)

`ARRAY_INGEST` runs `bin/epic_process.R` (minfi): reads the IDATs (auto-detects EPIC/450K), normalises (`preprocessFunnorm` for ≥ 2 samples, `preprocessNoob` for a single sample), drops probes failing a detection-p filter, extracts beta, and writes `epic_beta.tsv` + `epic_manifest.tsv` (+ `epic_qc.txt` recording array type, genome build, and probe counts). `ARRAY_TO_METHYL` (`methylforge array`, `array.py`) maps probes to genomic coordinates via the manifest, collapses duplicate positions, and writes one MethylForge per-cytosine `*.methyl.tsv` per sample (arrays carry no coverage, so a nominal `--cov` of 30 is assigned and documented in QC). Array samples then flow through the identical downstream as sequencing.

10.2 Stage 1 — Read QC & trimming (short)

`QC_TRIM` runs FastQC for a report, then **fastp** for gentle, bisulfite-appropriate quality/adapter trimming (`--detect_adapter_for_pe --qualified_quality_phred 20 --length_required 30`), preserving read structure. Output: trimmed R1/R2 and a `fastp.json`.

10.3 Stage 2 — Alignment

Short (`ALIGN_SHORT`). Default **Bismark** (bowtie2/HISAT2 backend): prepares the bisulfite genome once, aligns paired reads, then `deduplicate_bismark`. Alternative **bwa-meth** (`--aligner bwameth`): `bwameth.py` align → `samtools sort` → `samtools markdup`. Output: `<sample>.dedup.bam`.

Long (`ALIGN_LONG`). For native-5mC data, aligns with `minimap2 -ax map-ont -y` (the `-y` flag carries MM/ML modification tags through alignment; a modBAM is first converted to a tagged FASTQ), sorts/indexes, then `modkit pileup … --cpg --ref` to a per-CpG **bedMethyl**. Output: `<sample>.cpg.bedmethyl`.

10.4 Stage 3 — Methylation extraction (short)

`EXTRACT` runs **MethylDackel** on the dedup BAM (`--CHH --CHG` so downstream conversion-rate QC can use non-CpG contexts) to a per-CpG **bedGraph** carrying real methylated/unmethylated counts. Output: `<sample>.cpg.bedGraph`.

10.5 Stage 4 — Methylation QC

`qc.py::methylation_qc` computes, on the biological CpGs (spike-in contigs excluded): `n_cpg`, covered and callable CpG counts, mean/median CpG coverage, global methylation by context (CpG/CHG/CHH), **conversion efficiency** (spike-in → provided → CHH, in that order of preference, recorded in `conversion_source`), a spike-in block (lambda conversion, pUC19 methylation/over-conversion), and an **inferred sex** from the chrY/autosome coverage ratio. It applies the default gates and sets `qc_pass`:

Gate	Default	Meaning
`min_conversion`	0.98	conversion efficiency
`min_mean_cov`	10.0	mean CpG coverage
`min_cpg_callable`	1000	CpGs at $\geq$ `min_site_cov`
`min_site_cov`	5	per-site callable threshold
`min_puc19_meth`	0.95	methylated-control retention (over-conversion gate; only if pUC19 present)

Arrays legitimately fail the coverage/callable gates (nominal coverage) — interpret array QC on conversion-independent metrics.

10.6 Stage 5 — Imprinted-DMR analysis

`imprinting.py::analyse_imprinting` computes a **coverage-weighted mean methylation** over the CpGs in each panel region (requiring ≥ 3 covered CpGs to make a call) and calls `hypermethylation` ($\beta \geq 0.65$), `hypomethylation` ($\beta \leq 0.35$), or `normal`, attaching the direction-appropriate disorder. `summarise_imprinting` produces a headline and flags **MLID** when ≥ 2 loci are abnormal. Thresholds (`hyper`/`hypo`) are tunable.

10.7 Stage 6 — Differential methylation

`cohort.py::build_matrix` assembles a feature $\times$ sample beta matrix (per-sample coverage filter `min_cov`; keep a site only if covered in $\geq$ `min_samples_frac` of samples; optional restriction to a site list or region BED; CpG by default). `dmr.py::call_dmps` tests each CpG (`auto` = Mann-Whitney for ≥ 12/group, else Welch-t on M-values), applies BH-FDR, and flags DMPs at $|\Delta\beta| \geq$ `min_delta` and q $\leq$ `fdr`. `merge_dmrs` merges significant, same-direction DMPs within `max_gap` (default 1 kb) into DMRs with $\geq$ `min_cpg` (default 3) CpGs, reporting width, mean Δ, and minimum q-value. `annotate_regions` adds the nearest gene.

10.8 Stage 7 — Episignature classification

`episignature.py::classify` scores a query against class centroids. With scikit-learn installed it fits a linear **SVM** with probability calibration; otherwise it uses a **nearest-centroid correlation** model with temperature-scaled softmax pseudo-probabilities. A **positive** call requires the top-class probability ≥ 0.80 **and** a margin ≥ 0.15 over the runner-up; a control-class top $\rightarrow$ **control-like**; otherwise **inconclusive**. The verdict, top class, probability, margin, method, and per-class probabilities are returned; a positive disorder episignature is annotated as PS3-type functional support.

10.9 Stage 8 — Region annotation

`annotate.py::annotate_regions` assigns each DMR its nearest gene from a BED (`chrom start end name`) — distance 0 if overlapping, else the gap to the nearest edge. Dependency-free; in

a production Nextflow deployment this stage can instead call `annotatr`/a full GTF for CpG-island/promoter/regulatory context.

10.10 Stage 9 — Report engine and hand-off

`report.py::render_report` renders two self-contained HTML reports from one result set: a **clinical** report on the GBCF/Noguchi-branded template with the ACMG/AMP severity colour convention (Pathogenic = red, Likely-pathogenic = orange→red, VUS = amber, Benign = green) and embedded QC/imprinting/episignature figures; and a **research** report with full DMR detail and a volcano plot. It also writes a machine-readable `results.json`.

`export_excel.py` writes a per-sample **Excel findings workbook** (`<sample>.findings.xlsx`, sheets *Findings · Imprinting loci · QC*), tier-coloured with the same ACMG convention; `findings-xlsx` aggregates many samples into a cohort workbook. `export_familyforge.py` writes `<sample>.familyforge.json` (a `candidates.json`-compatible object whose `rows` carry FamilyForge's base keys plus epigenetic extras) and a tolerant `<sample>.epi_findings.tsv`. Findings are tiered **A** (diagnostic-grade imprinting defect or positive episignature), **B** (needs confirmation / DMR over a disease gene), **C** (research DMRs). Because the Excel, HTML and FamilyForge exports are built from the *same* tiered rows, they never disagree.

10.11 Batch cohort QC dashboard

`dashboard.py` aggregates per-sample QC across a run into one MultiQC-style self-contained HTML (`cohort_qc.html`) with overview plots (conversion, coverage, callable CpGs, global methylation, spike-in controls, pass/fail) and a sortable table with pass/fail badges, plus a `cohort_qc_summary.tsv`. It can collect QC from methylation files directly or from existing `results.json` files.

Outputs

For a `sequencing` run with `--outdir results`:

```
results/
  <sample>/
    qc/            <sample>_R{1,2}.trim.fq.gz, <sample>.fastp.json   (short-read)
    align/         <sample>.dedup.bam  (short) | <sample>.cpg.bedmethyl (long)
    methylation/   <sample>.cpg.bedGraph                            (short)
    report/        <sample>.clinical.report.html
                   <sample>.research.report.html
                   <sample>.results.json
                   <sample>.findings.xlsx
                   <sample>.familyforge.json
                   <sample>.epi_findings.tsv
  array/           epic_beta.tsv, epic_manifest.tsv, epic_qc.txt,
↪   methyl/*.methyl.tsv  (array mode)
  cohort_qc.html
  cohort_qc_summary.tsv
  pipeline_info/   execution_report.html, timeline.html, trace.txt
```

Key files:

- **`<sample>.results.json`** — the complete machine-readable result (QC metrics, imprinting table, episignature verdict, DMR summary, provenance).
- **`<sample>.familyforge.json`** — tiered findings for FamilyForge / LIMS ingest.
- **`<sample>.findings.xlsx`** — clinician-facing spreadsheet (*Findings · Imprinting loci · QC*).
- **`cohort_qc_summary.tsv`** — one QC row per sample for the whole run.

Quality control & interpretation

- **Conversion** is the first thing to check. Below `min_conversion` (0.98) apparent methylation is inflated; the report records the *source* of the estimate (spike-in > provided > CHH). Prefer spike-ins for a defensible number.
- **Coverage / callable CpGs** gate statistical power. Array data intentionally fails the coverage gate (nominal coverage) — judge arrays on conversion-independent metrics and probe counts.
- **Imprinting calls** hinge on the $\beta \geq 0.65$ / ≤ 0.35 thresholds around a per-locus normal of ~0.5; a **single** abnormal locus points to one disorder, $\geq$ **2** to MLID (which changes the differential). Confirm with an orthogonal assay (MS-MLPA, pyrosequencing) before reporting.
- **Episignatures** are only as good as the reference. An **inconclusive** verdict is a feature, not a failure — it means the profile did not confidently match any known signature. A **positive** verdict provides PS3-type functional support for a related variant but is not itself a diagnosis.
- **DMRs** from small cohorts are exploratory; the M-value Welch-t path exists so small studies produce interpretable p-values, but validate regions of interest.

Reproducibility & provenance

Every `run` records a provenance string (`methylforge run · <ISO-timestamp> · input=<file>`) in `results.json` and the FamilyForge export. Nextflow writes an execution report, timeline, and trace under `pipeline_info/`. Pin reproducibility by: committing the exact `env/*.yml` (tool versions are pinned), using a container profile (`singularity`/`apptainer`) on shared systems, keeping the episignature reference and imprinted-DMR panel under version control with their validation notes, and recording the reference genome build. The synthetic fixture uses fixed RNG seeds, so `testdata`/`selftest` are byte-reproducible.

Troubleshooting

Symptom	Cause	Fix
`error: provide --samplesheet …`	mode needs an input	pass `--samplesheet` (or `--idat_dir`/`--array_beta` for array)
Nextflow parse/syntax error on the modules	strict language parser	`export NXF_SYNTAX_PARSER=v1` before running
`ModuleNotFoundError: openpyxl`	core dep missing (old checkout)	it is now a core dependency — `pip install -e .` again
Heavy-env create fails on Apple Silicon	bioconda has no arm64 build	`CONDA_SUBDIR=osx-64 mamba env create -f env/align_short.yml`
Array run keeps hg19 coordinates	sesame registry stale, no chain	pass `--array_chain hg19ToHg38.over.chain`, or install `sesame/sesamedata`
Conversion efficiency is `null`	no spike-in and no CHH sites	add lambda/pUC19 spike-ins (`bin/prepare_reference.sh`) or provide a value
Every sample fails the coverage gate	array data (nominal coverage)	expected; judge arrays on conversion-independent metrics
Episignature always `inconclusive`	reference too small / not matched	build a larger, ancestry-matched reference; check CV metrics in `manifest.json`
SVM episignature behaves oddly on newer sklearn	`SVC(probability=True)` calibration changed (deprecated 1.9, removed 1.11)	the env pins `scikit-learn<1.11`; or omit sklearn to use the nearest-centroid default
DMR run: `need ≥1 case and ≥1 control`	groups map has no `case`/`control`	ensure the group column/`--groups` uses those labels
Bismark can't find the genome index	index not prepared	`bismark_genome_preparation <genome_dir>` once; MethylForge reuses it

Limitations & validation

MethylForge is **research software**. The bundled imprinted-DMR panel coordinates, the QC gate thresholds, and any episignature classifier are a curated **starting point** and must be locally validated — ideally against **ancestry-matched controls**, since public methylation references are overwhelmingly European-ancestry — before any diagnostic use. The synthetic fixture proves the machinery, not clinical performance. The Python annotation stage is deliberately lightweight (nearest-gene only); richer genomic-context annotation requires `annotatr`/a GTF. Long-read methylation depends on the quality of upstream modified-base basecalling, which MethylForge does not perform. Report any clinically actionable finding only after orthogonal confirmation and interpretation by a qualified professional.

Glossary

- **beta / β** — methylated fraction at a cytosine (0–1).
- **M-value** — logit of beta; the scale for parametric methylation testing.
- **CpG / CHG / CHH** — cytosine sequence contexts; CHH is a conversion-rate proxy.
- **DMP / DMR** — differentially methylated position / region.
- **ICR** — imprinting control region.
- **MLID** — multi-locus imprinting disturbance (≥ 2 abnormal imprinted loci).
- **Episignature** — a disorder-specific genome-wide methylation pattern.
- **Conversion efficiency** — fraction of unmethylated C correctly converted.
- **Over-conversion** — erroneous conversion of *methylated* C (false-negative rate).
- **PS3** — ACMG functional-evidence criterion a positive episignature can support.

Citation & the forge family

If you use MethylForge, please cite the repository (see `CITATION.cff`). MethylForge is the **epigenome** counterpart to **CallForge** (targeted-capture germline sequence variants) and **FamilyForge** (family-based interpretation and reporting): a methylation finding — an imprinting-locus defect or a positive episignature — can be carried into the same interpretation and clinical-report workflow as a sequence variant via the FamilyForge-compatible hand-off.

MethylForge — Bright Adu, Genomics & Bioinformatics Core, Noguchi Memorial Institute for Medical Research, University of Ghana, Legon, Ghana.